Desafio 1

Sistema 1 na árvore de comportamento

Um desafio de pesquisa ligado ao projeto: usar modelos de decisão rápida, do tipo "sistema 1", para comandar a navegação semântica do robô da CPTM, no lugar de partes ou de toda a árvore de comportamento do Nav2. É um tema aberto, com trabalhos publicados há semanas, e cabe num artigo curto.

O desafio

PERGUNTA CENTRAL

Um modelo de sistema 1 consegue tomar as decisões semânticas da navegação do robô (para onde ir, como se recuperar, quando pedir ajuda) com robustez igual ou melhor que a árvore de comportamento, sem pagar a latência de um LLM, e sabendo reconhecer quando não deve decidir sozinho?

Você escolhe um recorte (as sugestões de lacunas estão abaixo, mas recortes próprios são bem-vindos), implementa no simulador sobre o sistema que o seu grupo já está construindo, compara com uma linha de base e escreve o resultado como artigo. Mais importante que o resultado ser positivo é você entender e conseguir explicar cada decisão que tomou.

Contexto o que você precisa saber antes de começar

Sistema 1 e sistema 2. Kahneman [1] separa o pensamento rápido e intuitivo (sistema 1) do lento e deliberado (sistema 2). Em robótica, a ideia virou arquitetura: um modelo grande e lento planeja, e um modelo pequeno e rápido executa. No Helix [2] e no GR00T N1 [3], o sistema 1 é uma política visuomotora treinada ponta a ponta; em navegação por linguagem, o mesmo vale para [4].

Modelos "System One". O Jev (TypeSafe AI, acesso antecipado desde 15/09/2026) é outra coisa: não gera texto. Recebe um estado em texto ou JSON e um conjunto de perguntas tipadas (escolha entre opções, nota numérica, sim/não) e devolve probabilidades, todas numa única passada [5]. Não lê imagens, é acessado só por API e tem fraquezas declaradas pelo próprio fabricante [6]. O Laya [7] é uma alternativa aberta no mesmo estilo, que roda localmente.

Árvores de comportamento. O Nav2 [8] decide o que fazer (planejar, seguir o caminho, recuperar-se de uma falha) com uma árvore de comportamento [9]: um grafo de nós de condição, ação e controle, executado em ticks. As condições e seletores são regras escritas à mão, e é nelas que o sistema 1 pode entrar.

A diferença que torna o tema novo. Quase todo "sistema 1" da literatura de robótica é treinado sobre imagens. Um sistema 1 **simbólico e sem treino**, decidindo sobre o estado do Nav2, ainda não foi avaliado.

O que já existe situação em outubro de 2026

- **Decisões tipadas em árvores de comportamento.** SharedKV-BT [10] (05/10/2026) é o trabalho mais próximo: cada nó ativo pontua candidatos em paralelo. Mas usa um LLM comum, não um modelo de sistema 1; não usa ROS nem Nav2; acerta 1 de 10 tarefas de navegação; e os autores dizem que a confiança não serve para decidir quando se abster.
- **Jev com Nav2.** Existe um repositório [11] em que o Jev escolhe a recuperação do Nav2 num TurtleBot 3 simulado. Prova que a ideia é implementável, mas não tem métricas, linha de base nem artigo.

- **Jev em agentes.** [12] mostra que o Jev reduz custo e latência, mas o sucesso pode cair e o custo por tarefa bem-sucedida pode subir; os autores pedem métricas de recuperação. [13] analisa 2.170 projetos públicos com Jev: só 1,2% são de robótica e nenhum menciona ROS.
- **Calibração.** [14] acha bem calibradas as escolhas do Jev, mas ruim o limiar padrão das respostas sim/não; [15] precisou recalibrar; [16] encontra uma incerteza que o modelo não expõe.
- **LLMs e árvores.** Gerar a árvore com um modelo de linguagem já é feito [17]; escolher habilidades de uma árvore com um LLM embarcado também [18]. Os dois atuam no planejamento, não nas condições em tempo de execução.
- **Rápido e lento com gatilho de incerteza.** VerNav [19] só chama o modelo gerador quando a entropia passa de um limiar, mas em navegação visual, sem estado simbólico e sem garantia formal. KnowNo [20] dá garantia de cobertura para pedir ajuda, mas com LLMs.

Sugestões de lacunas

escolha uma, combine duas ou proponha outra

Cada sugestão é um recorte possível. As lacunas 1 e 3 se combinam bem num único artigo.

1 INTEGRAÇÃO

Nós de sistema 1 dentro da árvore do Nav2

Trocar **elementos** da árvore: condições e seletores "semânticos" (qual recuperação aplicar, o destino está bloqueado por multidão, vale ir ao ponto de interesse equivalente) passam a consultar um modelo de sistema 1, que escala para um LLM quando a confiança é baixa.

Pergunta

O ganho em robustez compensa a latência e o custo, comparado à árvore padrão e a um LLM em cada nó? Uma chamada de 70 a 500 ms não cabe no tick de 10 Hz da árvore: como projetar o nó assíncrono?

Como medir

Sucesso da missão, tempo até recuperar, colisões, latência p50/p95 e efeito na taxa de tick, custo por missão bem-sucedida [12], fração de decisões escaladas.

Leia: [8] [11] [12] [18]

2 ARQUITETURA

Sem árvore: um laço de decisão de sistema 1

Trocar a árvore **inteira** por um laço que, a cada passo, entrega ao modelo o estado do robô e o grafo topológico de pontos de interesse e pede a próxima ação de alto nível (ir ao nó X, esperar, pedir ajuda, replanejar). O Nav2 continua cuidando do movimento.

Pergunta

O que se perde e o que se ganha ao abrir mão da estrutura explícita da árvore (previsibilidade, verificação, facilidade de depurar) em troca da flexibilidade do modelo? Há uma forma híbrida melhor que as duas?

Como medir

As mesmas métricas da lacuna 1, mais tarefas fora do previsto pela árvore (cenários que ninguém programou) e esforço para adicionar um comportamento novo.

Leia: [1] [2] [3] [9]

3 CALIBRAÇÃO

Quando agir e quando perguntar

A probabilidade que o modelo devolve é confiável para decidir entre agir, perguntar ao passageiro ou escalar? Os primeiros trabalhos sobre o Jev discordam [14, 15, 16], e ninguém mediu isso em decisões de robô.

Pergunta

Auditar a calibração em decisões de navegação e serviço e aplicar predição conformal, no estilo de [20], para garantir uma taxa máxima de erro: se mais de uma opção é plausível, o robô pergunta.

Como medir

ECE, Brier, diagrama de confiabilidade, cobertura empírica contra a desejada, tamanho médio do conjunto ("taxa de ajuda"), sucesso no simulador com e sem o limiar conformal.

Leia: [14] [15] [16] [19] [20]

4 REPRESENTAÇÃO

O que o modelo enxerga

O Jev só lê texto e declara fraqueza com números, estado grande com distratores e ordem das opções [6]. Como transformar o estado do robô (pose, mapa de custo, progresso, pontos de interesse) num texto que o modelo decide bem?

Pergunta

Comparar serializações numérica, topológica (nó atual e vizinhos no grafo) e semântica ("corredor B, perto da escada"); medir a sensibilidade à ordem e ao nome das opções.

Como medir

Acurácia e calibração por tipo de serialização, variação da decisão sob permutação das opções, tamanho do estado em tokens.

Leia: [6] [13] [14]

5 LINGUAGEM

Navegação semântica a partir da fala do passageiro

"Me leva num lugar pra sentar perto da plataforma": escolher o ponto de interesse certo no grafo da estação a partir de um pedido aberto, em português. O sistema 1 escolhe entre os candidatos; o LLM só entra quando a escolha é incerta.

Pergunta

O Jev foi treinado principalmente em inglês [6]. Quanto ele perde em português coloquial, e o que um modelo aberto multilíngue no mesmo estilo [7] recupera?

Como medir

Acurácia de escolha do destino, taxa de escalonamento ao LLM, latência entre a fala e a escolha do destino, comparação com um LLM sozinho.

Leia: [6] [7] [13] [19]

Como avaliar o seu sistema

Qualquer que seja o recorte, o artigo precisa de uma comparação justa. Algumas peças que se repetem:

Peça	Sugestão
Linha de base	A árvore padrão do Nav2 (navegar com replanejamento e recuperação); um LLM no lugar do sistema 1; o sistema 1 sem limiar de escalonamento.
Cenários	Corredor bloqueado, robô deslocado ("sequestrado"), ponto de interesse interditado, multidão, passageiro que parou de seguir, destino ambíguo. Repita cada um várias vezes.
Métricas	Sucesso da missão, tempo, colisões, latência por decisão (p50 e p95), custo, fração de decisões escaladas, calibração (ECE, Brier).
Ablação	Tire uma peça de cada vez (o limiar, o estado topológico, o fallback ao LLM) e mostre o efeito.

Ferramentas e recursos

- **Jev** [5]: API da TypeSafe. O acesso é por lista de espera; peça cedo. Registre a versão do modelo em cada experimento, porque ela pode mudar no meio do trabalho.
- **Laya** [7]: alternativa aberta e local (inclusive multilíngue). Use como segundo backend, para que o trabalho seja reproduzível mesmo sem acesso ao Jev.
- **Nav2** (docs.nav2.org) e **BehaviorTree.CPP** (behaviortree.dev): é aqui que entram os nós novos.
- **Simulação**: o mesmo mundo e as mesmas plataformas que o seu grupo usa no projeto.
- **Ponto de partida**: o repositório [11] mostra uma integração mínima de Jev com Nav2.

Cuidados

- A área anda rápido: vários trabalhos citados têm semanas. Refaça a busca no arXiv ("Jev", "System One model", "behavior tree" + "LLM") no início e antes de entregar.
- Resultado negativo também é resultado: se o sistema 1 piorar a navegação, mostrar por quê é uma contribuição.
- Use IA para estudar e programar, mas você precisa conseguir explicar cada parte do que entregou.
- Não use materiais internos da CPTM no artigo nem em repositórios públicos.

Referências

[1] KAHNEMAN, D. Thinking, Fast and Slow. New York: Farrar, Straus and Giroux, 2011.

[2] FIGURE AI. Helix: A Vision-Language-Action Model for Generalist Humanoid Control. 2025. https://www.figure.ai/news/helix

[3] NVIDIA et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. arXiv:2503.14734, 2025.

[4] WEI, M. et al. Ground Slow, Move Fast: A Dual-System Foundation Model for Generalizable Vision-and-Language Navigation. arXiv:2512.08186, 2025.

[5] TypeSafe AI. Jev documentation: introduction. 2026. https://docs.typesafe.ai/introduction

[6] TypeSafe AI. Jev 1.13: model jaggedness. 2026. https://docs.typesafe.ai/model-jaggedness/jev-1.13

[7] Convai Innovations. Laya: open System One decision model (Apache 2.0). 2026. https://huggingface.co/convaiinnovations/laya

[8] MACENSKI, S. et al. The Marathon 2: A Navigation System. In: IEEE/RSJ IROS, 2020. arXiv:2003.00368.

[9] COLLEDANCHISE, M.; ÖGREN, P. Behavior Trees in Robotics and AI: An Introduction. CRC Press, 2018. arXiv:1709.00084.

[10] WAKE, N.; WAGLE, J. SharedKV-BT: Node-Local Typed Decisions for Behavior-Tree Agents. arXiv:2610.07327, 2026.

[11] MAHAJAN, P. JEV_SMARTROBOTCONTROL (repositório: Jev supervisionando o Nav2 num TurtleBot 3). 2026. https://github.com/mahajanparth/JEV_SMARTROBOTCONTROL

[12] LING, Y. et al. When Faster Decisions Do Not Make Better Agents: An Empirical Study of Jev in Agent Workflows. arXiv:2610.13092, 2026.

[13] LING, G.; XUE, M.; YE, Z. Jev in the Wild: A Data-Driven Analysis of the Jev Model's Functionality, Applications and Ecosystem. arXiv:2609.30216, 2026.

[14] DEUSSER, T.; SPARRENBERG, L.; SIFA, R. Evaluating and Benchmarking the System One Model Jev. arXiv:2609.37647, 2026.

[15] RAFE, A.; DAS, S. Calibrated Decisions at Scale: Converting Police Crash Narratives into Probabilistic Crash Variables with a System One Model (Jev). arXiv:2609.24052, 2026.

[16] PORCEDDA, R. Jev thinks "I don't know", but doesn't say it: Introducing Sys1Cal-v1 Dataset for Probability Calibration. arXiv:2609.35342, 2026.

[17] IZZO, R. A.; BARDARO, G.; MATTEUCCI, M. BTGenBot-2: Efficient Behavior Tree Generation with Small Language Models. arXiv:2602.01870, 2026.

[18] HÜBEL, F. et al. BT-Agent: Deploying Onboard Robot Agents with Behavior Trees and Multi-Fidelity Evaluation. arXiv:2610.13862, 2026.

[19] WANG, Z. et al. VerNav: Verifier-First Low-Latency Vision-and-Language Navigation. arXiv:2609.00920, 2026.

[20] REN, A. Z. et al. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In: CoRL, 2023. arXiv:2307.01928.